

T09 Security Testing

Quy trình kiểm thử an ninh cho EShop bằng SAST, DAST và AI triage có kiểm chứng.

Semgrep

OWASP ZAP

OpenAI / Gemini

DAST + SAST

Giảng viên hỗ trợ



Trần Duy Hoàng

Giảng viên chính



Hồ Tuấn Thanh

Trợ giảng

Lê Trung Kiên

MSSV 23127075

**Mai Thị Kim
Duyên**

MSSV 23127185

Lâm Hữu Khánh

MSSV 23127205

Lê Mai Hoài Bảo

MSSV 23127326

SEMINAR FLOW

SAST + DAST

Nội dung trình bày

Bốn phần chính đi từ khung kiểm thử tổng thể đến hai nhánh công cụ của nhóm.

01

Workflow

02

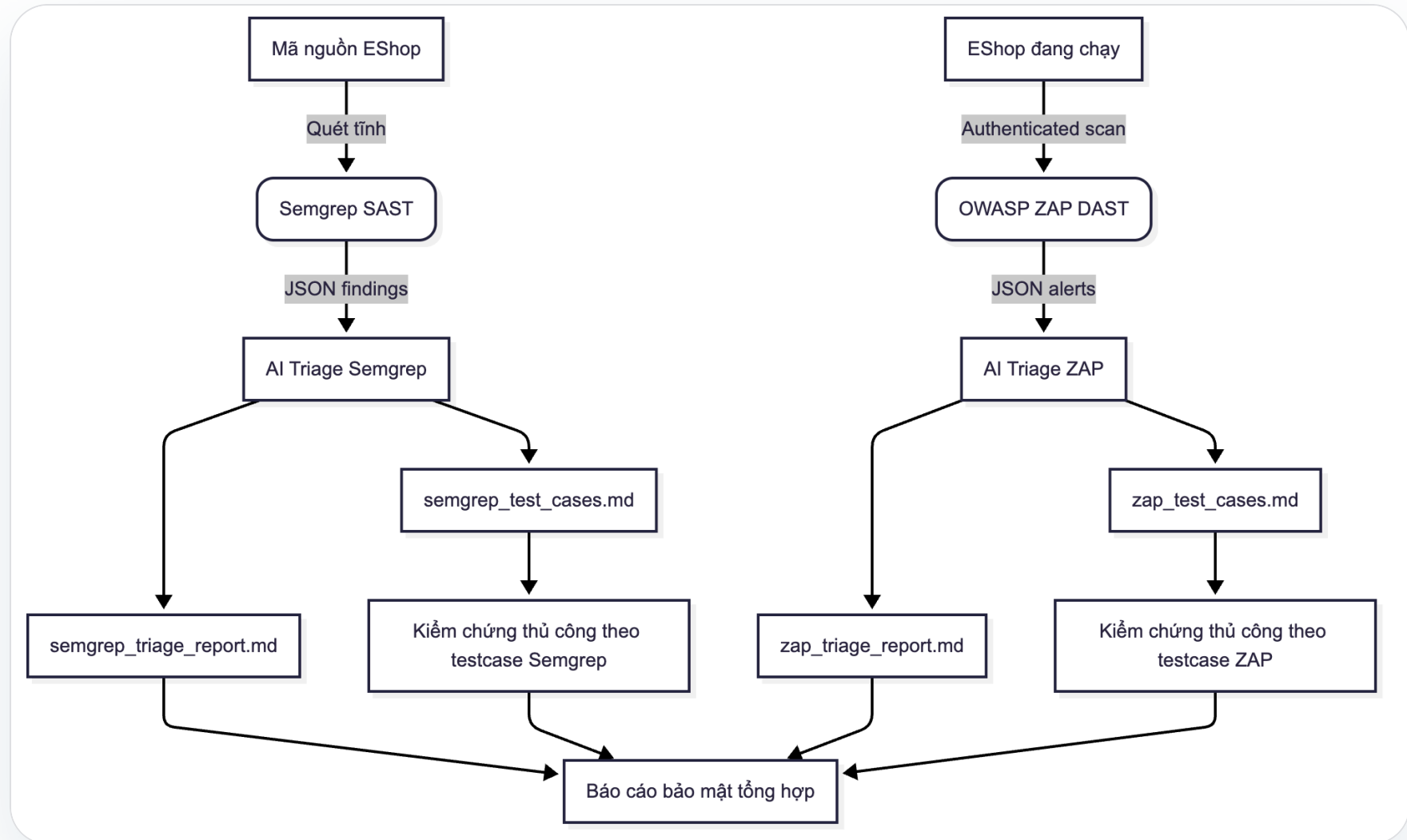
OWASP Top 10

03

SAST và Semgrep

04

DAST và ZAP



OWASP Top 10 2025 dùng làm khung chọn ruleset và scan policy.

A01

Broken Access Control

Kiểm soát quyền truy cập không chặt, user có thể truy cập dữ liệu hoặc chức năng ngoài quyền.

A02

Security Misconfiguration

Cấu hình server, framework, CORS, header hoặc môi trường triển khai không an toàn.

A03

Software Supply Chain Failures

Rủi ro từ thư viện, dependency, build pipeline, artifact hoặc thành phần bên thứ ba.

A04

Cryptographic Failures

Mã hóa, truyền dữ liệu, lưu secret hoặc bảo vệ dữ liệu nhạy cảm không đúng cách.

A05

Injection

Dữ liệu đầu vào bị đưa vào SQL, command, template hoặc interpreter mà không kiểm soát đúng.

A06

Insecure Design

Thiết kế nghiệp vụ hoặc kiến trúc thiếu cơ chế phòng vệ ngay từ đầu.

A07

Authentication Failures

Xác thực, token, session, reset password hoặc quản lý tài khoản có lỗi.

A08

Software or Data Integrity Failures

Không kiểm chứng tính toàn vẹn của dữ liệu, update, CI/CD hoặc code được nạp vào hệ thống.

A09

Logging and Alerting Failures

Thiếu log, monitoring hoặc cảnh báo khiến việc phát hiện sự cố bị chậm.

A10

Mishandling of Exceptional Conditions

Xử lý lỗi hoặc ngoại lệ không an toàn, làm lộ thông tin hoặc tạo trạng thái khó kiểm soát.

STATIC ANALYSIS

SEMGREP CLI

SAST và Semgrep

Phần này giới thiệu cách kiểm thử tĩnh mã nguồn bằng Semgrep, từ cài đặt đến scan và AI triage.

01

Giới thiệu về SAST và Semgrep

02

Cài đặt Semgrep

03

Hướng dẫn sử dụng Semgrep

04

AI triage cho Semgrep

SAST phát hiện rủi ro trước khi ứng dụng chạy.

SAST

Static Application Security Testing phân tích mã nguồn hoặc file cấu hình mà không cần chạy ứng dụng. Cách này phù hợp để phát hiện sớm hardcoded secret, API không an toàn, code injection, lỗi framework phổ biến và cấu hình rủi ro.

Semgrep

Semgrep là công cụ SAST chạy bằng CLI, dùng rule dạng pattern để tìm đoạn code có dấu hiệu nguy hiểm. Với đề T09, flow chính dùng ruleset p/owasp-top-ten, sau đó đưa output JSON vào script AI triage để sinh báo cáo kiểm chứng.

01

Source

Mã nguồn và file cấu hình EShop.

02

Ruleset

p/owasp-top-ten theo đề T09.

03

JSON

Giữ output scan gốc làm evidence.

04

AI triage

Sinh report để tester kiểm chứng.

Cài Semgrep CLI trước khi nói tới scan.

Phần triage dùng Python dependency riêng, còn việc quét source cần Semgrep CLI chạy được trong terminal.

Linux

```
python3 -m  
pip install  
semgrep
```

macOS

```
brew install  
semgrep
```

Windows

```
python -m pip  
install  
semgrep
```

Verify

Chạy semgrep -
-version trước
khi scan.

```
# Tạo môi trường cho script triage  
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install -r  
src/semgrep/requirements.txt  
  
# Cài và kiểm tra Semgrep CLI  
python3 -m pip install semgrep  
semgrep --version
```



```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r
src/semgrep/requirements.txt
```

```
# source mặc định
./eshop-sut
```

Scan đúng source root, loại bỏ dependency và build output.

- ☐ **Chạy từ root repo**
Seminar-SoftwareTesting là working directory chuẩn.
- ☐ **Source khác vị trí**
Đặt SOURCE_ROOT trỏ tới source EShop thật.
- ☐ **Không quét nhiều**
Exclude node_modules, dist, build, .next.

Lệnh chính tạo evidence gốc theo OWASP Top 10.

```
semgrep scan --config "p/owasp-top-ten" \  
  --exclude node_modules --exclude dist --exclude build --exclude .next \  
  --json -o src/semgrep/output/semgrep_results.json ./eshop-sut
```

File

src/semgrep/output/semgrep_res

Đọc nhanh

File, line, rule ID, severity, CWE và OWASP mapping.

Nguyên tắc

Giữ JSON gốc để đối chiếu với triage report.

Scan mở rộng tăng coverage, không thay thế flow chính.

Sau ruleset bắt buộc, nhóm có thể quét thêm theo công nghệ của EShop để phân tích sâu hơn.

p/nodejs

Backend Node.js và Express-like patterns.

p/javascript

JS security rules phổ biến.

p/react

Frontend React và request/browser patterns.

Custom rules

src/semgrep/rules/eshop-security.yml

Output: semgrep_results_extended.json

Coverage analysis

AI triage cần cấu hình rõ, không phải magic report.

```
AI_PROVIDER=openai-compatible  
AI_MODEL=google/gemini-2.5-flash-lite  
AI_MAX_TOKENS=1800  
OPENROUTER_API_KEY=sk-or-v1-...  
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
```

Không commit key

src/semgrep/.env
chứa API key thật.

Khi hết credit

Giảm

AI_MAX_TOKENS
hoặc chạy --
offline.

Từ JSON finding thô sang report kiểm chứng được.

01

Input

semgrep_results.json

02

Context

Script lấy source
context từ --
source-root.

03

Triage Report + Findings

semgrep_triage_report.
& folder findings/
— tổng hợp và
phân tích từng lỗi.

04

Test Cases

semgrep_test_cases.md
— kịch bản kiểm
chứng từng finding.

```
python src/semgrep/semgrep_ai_triage.py \  
  src/semgrep/output/semgrep_results.json \  
  --source-root eshop-sut \  
  --output-dir src/semgrep/output
```

12

findings hiện có

Hardcoded JWT secret trong backend/server.js và nhiều HTTP request không mã hóa trong frontend-mobile/App.js.

Đừng đọc nhãn AI như kết luận cuối.

- ☐ **Code tự viết hay dependency?**
Loại bỏ build output và thư viện ngoài scope.
- ☐ **Dữ liệu có tới sink nguy hiểm?**
Finding chỉ có giá trị khi đường đi runtime hợp lý.
- ☐ **Secret hoặc URL có dùng thật?**
Phân biệt production, local lab, sample và test helper.

Ví dụ testcase

Semgrep: hardcoded JWT secret.

RULE javascript.jsonwebtoken.security.jwt-hardcode.hardcoded-jwt-secret

FILE backend/server.js

CWE CWE-798

OWASP A07 Authentication Failures

TC-SEMGREP-001

Finding liên quan: SEMGREP-001

Mục tiêu test: Kiểm chứng khả năng giả mạo JWT khi SECRET_KEY bị hardcoded trong

Input

Source evidence: `const SECRET_KEY = "super_secret_key_that_should_not_be_here";, jwt.sign(..., SECRET_KEY), jwt.verify(..., SECRET_KEY).`

PoC: `src/semgrep/exploit.js`

01 Chạy PoC để tạo forged token.

```
node src/semgrep/exploit.js
```

02 Copy token sinh ra từ PoC.

03 Gửi request kiểm tra.

```
curl -i http://localhost:3000/api/users/me \
-H "Authorization: Bearer <forged_token>"
```

04 Ghi nhận status code, response body và quyền mà backend suy ra từ token.

Nếu còn lỗi

Nếu đã an toàn

source code.

AI TRIAGE EXTRACT

Backend chấp nhận
forged token, cho phép
truy cập như
user/admin hợp lệ.

Forged token bị từ
chối, backend trả
401/403 hoặc lỗi verify
JWT. 16/ 34

AI triage chỉ ra root cause và đường khai thác bằng forged JWT.

Trang thái: Chưa xem chứng

submission/User_Guide.md, Mục 10.1

01 Giải thích lỗ hổng

Khóa bí mật SECRET_KEY dùng để ký và xác thực
JWT được khai báo tĩnh trực tiếp trong mã
nguồn tại backend/server.js.

02 Proof of Concept

Khi có chuỗi
super_secret_key_that_should_not_be_here,
attacker có thể tự tạo JWT hợp lệ.

```
const jwt = require('jsonwebtoken');  
const LEAKED_SECRET =  
"super_secret_key_that_should_not_be_here";  
const payload = { id: 1, role: "admin" };  
const forgedToken = jwt.sign(payload, LEAKED_SECRET);  
console.log("Bearer " + forgedToken);
```

03 Mức độ ảnh hưởng

Authentication Bypass, Privilege Escalation, truy
cập, sửa đổi hoặc xóa dữ liệu trái phép nếu token
giả được backend tin tưởng.

04 Khuyến nghị khắc phục

Không lưu secret trong source code. Chuyển
SECRET_KEY sang process.env.JWT_SECRET,
không commit .env, và rotate toàn bộ token cũ
sau khi thay secret.

DYNAMIC TESTING

OWASP ZAP

RUNTIME EVIDENCE

DAST và ZAP

Phần này chuyển từ source code sang ứng dụng đang chạy, tập trung vào crawl, scan, report và AI triage cho ZAP.

01

DAST và OWASP ZAP

Khái niệm DAST, vai trò của ZAP và hai hướng chạy: GUI thủ công hoặc CLI tự động.

02

Cài đặt OWASP ZAP

Chuẩn bị ZAP GUI, Python client, daemon và cấu hình target qua `src/zap/.env`.

03

Chạy ZAP bằng GUI

Proxy trình duyệt, cấu hình context, login, Forced User Mode và kiểm tra evidence trong Alerts.

04

Chạy ZAP bằng CLI

Dùng `scan_zap.py` để spider, AJAX Spider, active scan và xuất JSON hoặc HTML.

RUNNING APPLICATION

REQUEST / RESPONSE

DAST kiểm thử bảo mật trên ứng dụng đang chạy thật.

DAST không cần đọc source code. Tool gửi request HTTP, quan sát response, crawl trang, chạy passive scan và có thể active scan bằng payload kiểm thử.

HTTP
requestCrawl
routeScan
responseRuntime
alert

05

AI triage cho ZAP report

Đọc report
JSON, gom
alert, tạo

06

Testcase mẫu ZAP

Dùng alert ^{18 / 34}
runtime từ ZAP
để dựng

ZAP

OWASP ZAP là công cụ DAST mã nguồn mở.

Trong project T09, ZAP là tool chính để thu request/response runtime và tạo alert dựa trên hành vi thật của EShop.

GUI

Chạy bằng GUI

Proxy trình duyệt, quan sát request/response, tạo context và cấu hình authentication thủ công.

CLI

Chạy bằng CLI

Dùng `src/zap/scan_zap.py` để tự động hóa scan, xuất report JSON/HTML và đưa JSON vào AI triage.

JAVA 17+ REQUIRED

DESKTOP GUI

CROSS PLATFORM

Cài đặt ZAP GUI trên Windows, macOS và Linux.

ZAP GUI cần Java 17+ độc lập (Windows/Linux) hoặc tự động bundle sẵn (macOS). Lựa chọn phương thức Installer (.exe/.dmg/.sh) hoặc Package Manager phù hợp.

Windows Java 17+ độc lập

1. Installer (.exe)

Tải *Windows (64) Installer* từ [zaproxy.org/download](https://www.zaproxy.org/download)

. Mở setup wizard > Accept > Standard setup.

2. Package Manager

```
winget install --id=ZAP.ZAP -e
```

```
scoop install zaproxy
```

macOS Đã bundle Java

1. Homebrew Cask (Nhanh nhất)

```
brew install --cask zaproxy
```

2. macOS Installer (.dmg)

Tải file .dmg đúng chip Mac: Intel (*amd64*) hoặc Apple Silicon (*aarch64*).

Linux Java 17+ Full Desktop

1. Official Repos (Flatpak / Snap)

```
flatpak install flathub org.zaproxy.ZAP
```

```
sudo snap install zaproxy --classic
```

2. Package thủ công (.sh / .tar.gz)

```
chmod +x ZAP_*_Linux_Installer.sh && ./ZAP_*_Linux_I
```

GUI scan không login dùng cho endpoint public và route chưa cần token.

Flow này phù hợp để demo Quick Start, xem Sites, History, Alerts và evidence request/response.

Backend

<http://localhost:3080>

User FE

<http://localhost:5173>

Admin FE

<http://localhost:5173>

SPA

Dùng thêm Manual Explore hoặc AJAX Spider.

```
installer.sh
```

```
tar -xf ZAP_*_Linux.tar.gz  
&& ./zap.sh
```

⚠ Lỗi GUI: Dùng *java-17-openjdk* (bản GUI, không chọn *headless*).
Trên Wayland:

Mở ZAP GUI

- 01 Chọn không lưu session nếu demo nhanh, hoặc lưu session nếu cần evidence.

Quick Start

- 02 Vào Automated Scan, nhập target public và nhấn Attack.

Đọc kết quả

- 03 Xem URL trong Sites, request runtime trong History và lỗi trong Alerts.

Kiểm chứng evidence

- 04 Mở Request/Response của từng alert trước khi kết luận lỗi.

FIREFOX

LOCALHOST:8080

Browser proxy giúp ZAP bắt traffic thật khi người dùng thao tác.

Cấu hình Firefox đi qua ZAP proxy để ghi nhận request/response phát sinh từ frontend React và API backend.

Xác nhận proxy trong ZAP

01 Tools, Options, Network, Local Servers/Proxies, main proxy là localhost:8080.

Cấu hình Firefox

02 Settings, Network Settings, chọn manual proxy với HTTP proxy localhost, port 8080.

Bắt traffic localhost

03 Nếu traffic local không hiện, đặt `network.proxy.allow_hijacking_localhost=true` trong `about:config`.

Lưu evidence

04 Đối chiếu Sites, History, Request và Response trước khi chạy scan sâu hơn.

Authenticated GUI scan cần context, login request và Forced User Mode.

Flow này dùng cho giỏ hàng, profile, user API hoặc admin API, nơi request phải có session hợp lệ.

Context

Đưa frontend và backend vào cùng scope kiểm thử.

Login

Flag POST
/api/login làm
JSON-based
Auth Login
Request.

Users

Tạo user test, nhập
username/password

Active Scan

Chỉ chạy khi
scope đúng và
tài khoản test

Đăng nhập qua proxy

01 Mở EShop trong Firefox đã proxy, dùng tài khoản test để ZAP ghi nhận request login.

Kiểm tra auth

02 Xác nhận login URL, method, JSON payload và dấu hiệu đăng nhập thành công.

Crawl trong session

03 Chạy Spider hoặc AJAX Spider trên context, ưu tiên AJAX Spider cho frontend SPA.

Lưu kết quả

04 Chụp Sites, request login, request /api/users/me, Alerts và evidence quan trọng.

CLI flow cần Python client và ZAP daemon chạy riêng.

Script scan không tự mở giao diện ZAP. Nó kết nối tới daemon tại `http://localhost:8090`, rồi điều khiển spider, passive scan, active scan và export report.

Virtual env

Cô lập package cho repo, tránh cài global vào system Python.

Python client

Cài `python-owasp-zap-v2.4` để gọi ZAP API từ script.

Docker daemon

Chạy ZAP headless, expose API port 8090.

API key

Lab dùng `api.disablekey=true` để đơn giản hóa demo.

```
python3 -m venv venv
source venv/bin/activate
pip install python-owasp-zap-v2.4
```

```
docker run --rm -d --name eshop-zap --network
host \
  -v $(pwd):$(pwd) \
  ghcr.io/zaproxy/zaproxy:stable zap.sh -
daemon \
  -port 8090 -host 0.0.0.0 -config
api.disablekey=true
```

```
cp src/zap/.env.example src/zap/.env

ZAP_TARGET=http://localhost:3000
ZAP_URL=http://localhost:8090
ZAP_AUTH_ROLE=none
ZAP_MAX_URLS=300
ZAP_REPORT_FORMAT=json
ZAP_REPORT_FILE=src/zap/output/backend
_basic.json

ZAP_USER_EMAIL=test@eshop.com
ZAP_USER_PASSWORD=Test1234!
ZAP_ADMIN_EMAIL=admin@eshop.com
ZAP_ADMIN_PASSWORD=Admin123!
```

Cấu hình quyết định target, role, giới hạn URL và output.

Target

3000 cho backend, 5173 cho user frontend, 5174 cho admin frontend.

URL budget

ZAP_MAX_URLS chặn active scan khi SPA crawl quá nhiều URL.

Output

json cho pipeline AI, html cho đọc thủ công.

Credential

Điền user/admin test khi scan có authentication, không commit file .env thật.

Public CLI scan dùng cho backend API hoặc frontend chưa login.

Backend public scan thường đủ cho API runtime. Với frontend React/SPA, bật AJAX Spider để ZAP thấy route và request sinh ra sau khi JavaScript chạy.

Backend

backend_basic

Frontend SPA

Thêm --ajax-spider trước khi export JSON.

Report format

JSON cho AI triage, HTML cho đọc tay.

```
python3 src/zap/scan_zap.py \
  --target http://localhost:3000 \
  --report-format json \
  --output-file
src/zap/output/backend_basic.json
```

```
python3 src/zap/scan_zap.py \
  --target http://localhost:5173 \
  --ajax-spider \
  --report-format json \
  --output-file
src/zap/output/frontend_public_basic.json
```

```
python3 src/zap/scan_zap.py \  
  --target http://localhost:5173 \  
  --auth-role user \  
  --forced-user \  
  --ajax-spider \  
  --report-format json \  
  --output-file  
src/zap/output/frontend_user_basic.js  
on
```

```
python3 src/zap/scan_zap.py \  
  --target http://localhost:5174 \  
  --auth-role admin \  
  --forced-user \  
  --ajax-spider \  
  --report-format json \  
  --output-file  
src/zap/output/frontend_admin_basic.js  
on
```

Authenticated CLI scan phải chạy theo role riêng.

User frontend

Scan route user sau login bằng --auth-role user.

Admin frontend

Scan target 5174 bằng --auth-role admin.

Forced user

Bật --forced-user để request đi dưới user đã cấu hình.

401/403

Kiểm tra credential, /api/login và /api/users/me.

Bảng tổng hợp các flag CLI chính trong scan_zap.py

FLAG	Ý NGHĨA	KHI DÙNG	OUTPUT / HÀNH VI
--target	URL đích cần scan	Backend 3000, user FE 5173, admin FE 5174	Phân loại report theo target
--auth-role	Chọn tài khoản seed auth	none (khách), user, admin	Tạo auth context & token
--forced-user	Bật Forced User Mode	Scan endpoint yêu cầu đăng nhập	Gắn user vào mọi request scan
--ajax-spider	Crawl route sinh bởi JavaScript	Frontend Single Page App (React)	Bắt thêm route frontend
--scan-mode	Chế độ active scan	basic (mặc định) hoặc owasp-top10-2025	Chỉ bật scanner OWASP Top 10 2025
--max-urls	Giới hạn URL trước active scan	Tránh quá tải RAM khi SPA crawl rộng	Giữ spider/passive, bỏ active nếu vượt
--report-	Định dạng file xuất ra	json (cho AI triage)	Xuất file tại src/zap/output/

AI triage biến alert JSON thành report có thể kiểm chứng.

Flow ZAP tách riêng với Semgrep để giữ evidence runtime, endpoint và testcase theo đúng ngữ cảnh scan.

Input

Một hoặc nhiều report JSON từ ZAP: backend, frontend user, frontend admin.

zap_triage_report.md

Gom theo alert group, kèm endpoint bị ảnh hưởng và runtime evidence đại diện.

zap_test_cases.md

Sinh testcase cho từng endpoint hoặc từng request instance.

alerts/

Lưu prompt tiếng Việt và output AI tương ứng cho từng alert group.

Cấu hình một AI provider trong src/zap/.env.

OpenAI

```
OPENAI_API_KEY=your_openai_key  
OPENAI_MODEL=gpt-4.1-mini  
OPENAI_BASE_URL=https://api.openai.com/  
v1
```

OpenRouter

```
OPENROUTER_API_KEY=your_openrouter_key  
OPENROUTER_MODEL=google/gemini-2.5-  
flash  
OPENROUTER_BASE_URL=https://openrouter.  
ai/api/v1  
OPENROUTER_MAX_TOKENS=4096
```

Không commit API key thật vào repository. Nếu thiếu key, dùng --offline để tạo prompt và skeleton report trước.

Chạy triage trên nhiều report và lọc đúng target bằng prefix.

```
python3 src/zap/zap_ai_triage.py \  
  src/zap/output/backend_basic.json \  
  src/zap/output/frontend_user_basic.json \  
  src/zap/output/frontend_admin_basic.json \  
  --output-dir src/zap/output/markdown \  
  --target-prefix http://localhost:3000 \  
  --target-prefix http://localhost:5173 \  
  --target-prefix http://localhost:5174
```

JSON reports

Input positional,
không dùng --input.
Có thể truyền một
hoặc nhiều report.

--output-dir

Thư mục sinh
zap_triage_report.md,
zap_test_cases.md và
alerts/.

--target-prefix

Lọc đúng scope
backend, frontend
user, frontend admin.
Có thể truyền nhiều
lần.

--offline

Không gọi AI
provider, chỉ tạo
prompt, skeleton AI
output và Markdown
testcase.

Testcase ZAP replay request runtime để xác nhận alert còn đúng.

TC-ZAP-001

CONCLUSION

Alert liên quan: ZAP-001.

Mục tiêu: kiểm chứng CSP:

Failure to Define
Directive with No
Fallback.

Evidence đại diện:

Content-Security-
Policy: default-src
'none'.

Source JSON:

backend_basic.json.

```
GET http://localhost:3000
HTTP/1.1
host: localhost:3000
user-agent: Mozilla/5.0 ...
pragma: no-cache
cache-control: no-cache
Authorization: Bearer
<token_da_zap_ghi_nhan>
```

Thao tác kiểm chứng

32 / 34

- 1 Đảm bảo EShop và ngữ cảnh auth tương ứng đang chạy.
- 2 Chuẩn bị token, cookie hoặc session nếu request có header xác thực.
- 3 Gửi request tới URL và headers mà ZAP đã ghi nhận.
- 4 Ghi nhận status code, response headers và response body.
- 5 So sánh kết quả mới với evidence trong triage report.

Evidence before conclusion

Evidence đến từ tool, nhưng kết luận phải đến từ kiểm chứng.

SAST chỉ ra rủi ro trong source code. DAST xác nhận hành vi runtime. AI giúp tăng tốc đọc report, nhưng tester vẫn chịu trách nhiệm xác nhận bằng evidence.

SAST + DAST + AI Triage

User_Guide.md section 10.2

Expected: còn lỗi nếu response vẫn có evidence CSP

Cảm ơn mọi người đã theo dõi

Nhóm 06 / KDBK

Q&A

OWASP

ZAP

SEMGREP

Nguồn tham khảo

Các tài liệu được dùng để tổng hợp phần DAST, SAST, OWASP Top 10, ZAP và Semgrep.

01

Cloudflare OWASP Top 10

<https://www.cloudflare.com/learning/security/threats/owasp-top-10/>

02

OWASP Top 10 2025

<https://owasp.org/Top10/2025/>

03

ZAP Getting Started

<https://www.zaproxy.org/getting-started/>

04

ZAP API Docs

<https://www.zaproxy.org/docs/api/>

05

ZAP Video Playlist

<https://www.youtube.com/watch?v=dkUBYKFIUqE&list=PLavGKPt0xAp0jKJ1VM6qWVJmLsiG1NbPD&pp=0gcJCfICOCosWNin>

06

Semgrep Writing Rules

<https://semgrep.dev/docs/writing-rules/>

07

PortSwigger SAST Knowledge Base

<https://portswigger.net/kb/sast>